PROJECT HANDOVER DOCUMENT

Scipher Guardz — SecOps Dashboard

Version 2.9.20226 | April 2026
Vestas / Scipher Security Operations

CONFIDENTIAL — INTERNAL USE ONLY

1. Project Overview

Scipher Guardz is an internal Security Operations (SecOps) web dashboard built for Vestas / Scipher security teams. It provides a single-pane-of-glass view of the organisation's security posture, aggregating data from multiple sources including SecurityScorecard, Excel-based trackers, VAPT reports, patch data, and embedded monitoring tool dashboards (Datadog WAF, Dynatrace, Wazuh, Snyk, SonarQube, Jenkins).

Application Name	Scipher Guardz
Current Version	3.0.20426
Framework	Python / Flask
Database	PostgreSQL (Azure hosted + local fallback)
Auth	Flask-Login (session-based, bcrypt passwords)
Production Host	Linux server — /opt/secops-dash/
Production Port	443 (HTTPS via TLS/SSL)
Process Manager	systemd service: secops-dash.service
Support Email	Scipher.Support@Vestas.com
Source Path	C: \prod-apps\secops-dashboard

2. Repository & Directory Structure

The project does not use a hosted git provider — a local git history file is included (archive/git). The active codebase is under the root secops-dashboard/ folder. The archive/ folder contains previous iterations (v1 and v2) and can be ignored.

2.1 Top-Level Layout

```
secops-dashboard/
├── app/                      # Main Flask application package
│   ├── __init__.py           # App factory (create_app)
│   ├── config.py             # DB connectivity + fallback logic
│   ├── models.py             # SQLAlchemy User model
│   ├── static/               # CSS, JS, logos
│   │   ├── css/style.css
│   │   ├── js/scripts.js
│   │   ├── logos/            # Tool logos (Datadog, Wazuh, Snyk, etc.)
│   │   └── sources/          # *** UI DATA FILES (Excel, images) ***
│   └── main/                 # Main blueprint
│       ├── routes.py         # All URL routes
│       ├── bell.py           # Notification context processor
│       ├── errors.py         # HTTP error handlers
│       ├── file2data.py      # Excel ingestion helpers
│       ├── forms.py          # WTForms definitions
│       ├── health.py         # /health_api blueprint
│       └── templates/        # Jinja2 HTML templates
│           ├── base.html     # Master layout
│           ├── home.html     # Homepage / dashboard
│           ├── dashboard.html  # Embedded tool dashboards
│           ├── reports.html    # File browser
│           ├── full_report.html# SSC detailed report
│           ├── tools.html      # Tools listing
│           ├── changelog.html
│           ├── health.html
│           ├── errors/       # 400,401,403,404,405,413,429,500
│           ├── macros/       # Reusable Jinja2 macros
│           └── user/         # Auth templates (login, UM, settings)
├── run.py                    # Dev/STG entry point (HTTPS on :5001)
├── wsgi.py                   # WSGI entry point for Gunicorn
├── gunicorn_config.py        # Gunicorn bind + worker config
├── seeder.py                 # DB schema + default admin seeder
├── requirements.txt          # Python dependencies
├── .env                      # Environment variables (secrets)
├── changelog.md              # Version history
├── prod/secops-dash.service # systemd service unit
└── archive/                  # Old versions — ignore
```

2.2 Data Files (Critical)

The application reads live data from flat files rather than a database (except for user accounts). These files MUST exist on the server for the homepage to load correctly

File	Location	Purpose
ec2-patch.xlsx	app/static/sources/	EC2 OS patch status (multi-sheet)
projects.xlsx	app/static/sources/	Security projects underway
Accomplished_Activities.xlsx	app/static/sources/	Completed activities (multi-sheet)
vapt-img/*.png|jpg	app/static/sources/vapt-img/	VAPT scan result images
(Drive reports)	source/drive/	Files browseable in /reports route

Note: The note.txt file clarifies two source paths:

- app/static/sources/ — UI data consumed directly by the homepage
- source/drive/ — files served via the /reports file-browser (3 levels up from app/)

3. Tech Stack & Key Dependencies

3.1 Core Stack

Language	Python 3.12 / 3.13 (both .pyc caches present)
Web Framework	Flask (Blueprints pattern)
ORM	Flask-SQLAlchemy + SQLAlchemy
Database	PostgreSQL — psycopg2 / psycopg2-binary driver
Auth	Flask-Login + Werkzeug password hashing (bcrypt)
Forms	Flask-WTF / WTForms
Excel Parsing	pandas + openpyxl
PDF Preview	PyMuPDF (fitz)
DOCX Preview	mammoth
Markdown	markdown (Python)
HTTP Client	requests (for SecurityScorecard API)
Real-time	Flask-SocketIO + python-socketio (eventlet)
Scheduling	APScheduler
Caching	Flask-Caching
Rate Limiting	Flask-Limiter
Migrations	Flask-Migrate (Alembic)
WSGI Server	Gunicorn (production)
Env Vars	python-dotenv

3.2 Installation

```
# Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate          # Linux / macOS
# .\venv\Scripts\activate         # Windows

# Install dependencies
pip install -r requirements.txt
```

4. Configuration & Environment Variables

All configuration lives in the .env file at the project root. This file is NOT committed to source control and must be provisioned manually on any new server.

Variable	Example / Default	Description
APP_NAME	Scipher Guardz	Injected into all templates as app title
APP_VERSION	2.9.20226	Displayed in footer
MAINTENANCE_MODE	false	Set true to redirect all users to /maintenance
MAINTENANCE_WINDOW	ISO 8601 interval	Informational — displayed during maintenance
SUPPORT_EMAIL	Scipher.Support@Vestas.com	Shown in error pages and footer
session_secret_key	(strong random string)	Flask session signing key — MUST be secret
POSTGRES_URI	postgresql://user:pass@host/db	Primary Azure PostgreSQL connection string
POSTGRES_local_URI	postgresql://postgres:...	Local PostgreSQL fallback (auto-selected if Azure unreachable)
schema_name	secops_db	PostgreSQL schema used for all tables
DASHBOARD_URLS	JSON array of {title, url}	Embedded iframe dashboards on /dashboard page (e.g. Datadog WAF)
DOMAIN	utopusinsights.com,scipher.io	Comma-separated domains for SecurityScorecard API lookups
SSC_API_KEY	D3vbxd...	SecurityScorecard API token — rotate regularly

5. Database

5.1 Connection & Fallback Logic

config.py implements an automatic failover: on startup it attempts a 3-second TCP connection to the Azure PostgreSQL instance (POSTGRES_URI). If that fails it falls back to the local PostgreSQL instance (POSTGRES_local_URI). The chosen URI is exported as POSTGRES_URI and used by both SQLAlchemy and the raw SQLAlchemy engine in routes.py.

5.2 Schema

All tables live in the secops_db schema (configurable via the schema_name env var). Currently only one table exists:

Column	Type	Constraints	Notes
user_id	UUID	PK, NOT NULL	Auto-generated uuid4
email	String	UNIQUE, NOT NULL	Login identifier
password	String	NOT NULL	Werkzeug bcrypt hash
role	String	Indexed	ADMIN or USER (uppercased on write)
user_created_on	DateTime	NOT NULL	IST timezone, no microseconds
last_modified_on	DateTime	—	Auto-updated on record change
is_active	Boolean	NOT NULL	False = login blocked
theme	String(10)	—	system | light | dark
file_permission	String(10)	—	none | basic | view | write

5.3 Initializing the Database (Seeder)

Run seeder.py once on a fresh install to create the schema, tables, and a default admin account:

```
cd /opt/secops-dash
source venv/bin/activate
python seeder.py

# Default admin credentials created by seeder:
#   Email:    admin@mail.com
#   Password: password123   ← CHANGE in Production to a secure string
```

6. Application Architecture

6.1 App Factory

app/__init__.py exports create_app(), a standard Flask application factory. It loads env, initializes SQLAlchemy and Flask-Login, registers three Blueprints (main, errors, health), and injects app_name, app_version, and support_email as Jinja2 context globals.

6.2 Blueprints

Blueprint	Prefix	Responsibility
main	(none)	All primary routes — home, auth, user management, reports, file manager, SSC API, tools, dashboard, changelog
errors	(none)	HTTP error handlers for 400, 401, 403, 404, 405, 413, 429, 500
health	(none)	GET /health_api — JSON liveness check (UI + DB)

6.3 URL Route Map

Route	Auth?	Description
GET /	Public	Homepage — SSC scores, VAPT images, EC2 patch, projects, activities
GET /login POST /login	Public	Login form + submission
GET /logout	Required	Clears session, redirects to home
GET /dashboard	Public	Embedded iframe dashboards (Datadog WAF etc.)
GET /tools	Public	Links/tabs to monitoring tools
GET /reports/ [path]	Required	File browser over source/drive/
GET /view/<path>	Public	Serve PDF file directly
GET /data/<filename>	Public	Serve UI source data files
GET /full-report/<domain>	Required	SSC factor report for domain
GET /full-reports/<domain>	Required	Alternative SSC full report endpoint
GET /changelog	Required	Renders changelog.md as HTML
GET /health_api	Public	JSON: {UI: ok, database: ok/error}
GET /health	Public	Health status HTML page
GET /UM	Admin	User management list with pagination + search
POST /create	Admin	Create new user account
POST /<uuid>/edit	Admin	Edit user role, active status, file_permission
POST /<uuid>/delete	Admin	Delete user
POST /<uuid>/toggle	Admin	Toggle user active status

Route	Auth?	Description
POST /<uuid>/reset_password	Admin	Admin resets any user's password
GET+POST /reset-password	Public	Self-service password reset (no email token)
GET+POST /settings	Required	User theme preference (system/light/dark)

7. User Roles & Permissions

7.1 Roles

Role	Access
ADMIN	Full access: all routes, User Management (create/edit/delete/reset passwords), all file manager operations
USER	Standard authenticated access: home, reports, tools, dashboard, changelog, settings, file manager (subject to file_permission)

7.2 File Permissions (file_permission column)

Value	Capabilities
none	Cannot access /api/tree or /file-manager/list — file manager is completely hidden
basic	Can download files only
view	Can list, preview, download, and upload files
write	Full file manager: list, preview, download, upload, edit, and delete files

8. External Integrations

8.1 SecurityScorecard (SSC) API

The homepage fetches security grade data for configured domains from the SecurityScorecard public API. Results are cached in-memory for 24 hours (SSC_CACHE_TTL) to avoid excessive API calls.

API Base URL	https://api.securityscorecard.io/companies/{domain}
Auth Header	Authorization: Token {SSC_API_KEY}
Cache TTL	24 hours (in-memory, resets on process restart)
Domains	Configured via DOMAIN env var (comma-separated)
Error handling	On failure: returns cached data if available, otherwise shows N/A grade

8.2 Embedded Dashboards

The /dashboard page renders iframes for external monitoring tools. URLs are configured in the DASHBOARD_URLS env var as a JSON array:

```
DASHBOARD_URLS=[
  {"title": "WAF Dashboard",
"url": "https://p.datadoghq.eu/sb/402b5da2-..."}
]
```

8.3 Tool Logos (Static Assets)

The /tools page and base template reference logos for: Datadog, Dynatrace, Jenkins, Snyk, SonarQube, Wazuh, Vertex AI. All logos are in app/static/logos/.

8.4 AWS (boto3)

boto3 is listed as a dependency and imported in routes.py, but no active AWS calls are present in the current codebase. It appears to be scaffolded for future S3 integration.

9. Notification System (bell.py)

The bell.py module injects a notifications list into every template via a context processor. Currently the notifications are hardcoded placeholders:

```
notifications = [
  {"message": "🔔 Security alert on host 10.0.1.14", "link": ""},
  {"message": "📦 New VAPT report uploaded",         "link": "/reports/vapt"},
  {"message": "🛠 Maintenance window scheduled",     "link": "/maintenance"},
]
```

⚠ Action Required: The bell.py - 'actual logic to be pulled for notifications'. This module needs to be Completed and connected to a real data source (database table, webhook, or polling job) before the notification bell in the UI reflects live data.

10. Production Deployment

10.1 Server Layout

Deploy path	/opt/secops-dash/
Venv	/opt/secops-dash/venv/
Entry point	/opt/secops-dash/run.py
Startup script	/opt/secops-dash/SecopsDashboard.sh
Systemd unit	/etc/systemd/system/secops-dash.service
Logs dir	/var/log/secops-dash/
Log rotation	Daily — file named DDMMYY-secops-dash.log
SSL certs	/opt/ssl-certs/fullchain.crt, scipher.secops.com.key, chain.crt
Run user	sdnch

10.2 systemd Service

The service unit runs run.py directly via the venv Python (not Gunicorn) on port 443 with TLS. There is also a prod/secops-dash.service that uses Gunicorn — the Gunicorn version binds to 127.0.0.1:5000 and would require a reverse proxy (nginx). The currently active deployment uses the direct Python/SSL approach.

```
[Unit]
Description=secops-dashboard (HTTPS on 443)
After=network.target

[Service]
Type=simple
User=sdnch
WorkingDirectory=/opt/secops-dash
Environment=PYTHONUNBUFFERED=1
ExecStart=/opt/secops-dash/SecopsDashboard.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

10.3 Startup Script (SecopsDashboard.sh)

```
#!/bin/bash
LOGDIR="/var/log/secops-dash"
DATE=$(date +"%d%m%y")
LOGFILE="${LOGDIR}/${DATE}-secops-dash.log"
mkdir -p "$LOGDIR"
exec /opt/secops-dash/venv/bin/python3 /opt/secops-dash/run.py >>"$LOGFILE" 2>&1
```

10.4 Port 443 Capability

Because port 443 is a privileged port (<1024), the venv Python binary is granted the net_bind_service Linux capability instead of running as root:

```
sudo setcap 'cap_net_bind_service=+ep' /opt/secops-dash/venv/bin/python3

# Verify:
getcap /opt/secops-dash/venv/bin/python3
```

10.5 Service Management Commands

```
# Reload systemd after editing the .service file
sudo systemctl daemon-reload

# Start / restart / stop / status
sudo systemctl restart secops-dash
sudo systemctl status  secops-dash
sudo systemctl stop    secops-dash

# Enable on boot
sudo systemctl enable  secops-dash

# Tail today's logs
tail -n 50 /var/log/secops-dash/$(date +"%d%m%y")-secops-dash.log

# Confirm port 443 is bound
sudo lsof -i:443
```

11. Fresh Install Guide

Step 1 — Prerequisites

- Python 3.12 or 3.13
- PostgreSQL (local or Azure — at least one must be reachable)
- Git (optional — for version tracking going forward)

Step 2 — Copy Files

```
# Copy project to server
cp -r secops-dashboard/ /opt/secops-dash/
cd /opt/secops-dash
```

Step 3 — Virtual Environment & Dependencies

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Step 4 — Configure Environment

```
# Copy and edit the .env file
cp .env .env.bak        # keep original as reference
nano .env               # update all credentials

# Required fields to update:
# - session_secret_key  -> generate a strong random string
# - POSTGRES_URI        -> live Azure DB connection
# - SSC_API_KEY         -> SecurityScorecard API token
# - DOMAIN              -> comma-separated domain list
# - DASHBOARD_URLS      -> embedded dashboard iframe URLs
```

Step 5 — SSL Certificates

```
# Place certs at:
sudo mkdir -p /opt/ssl-certs
# /opt/ssl-certs/fullchain.crt
# /opt/ssl-certs/scipher.secops.com.key
# /opt/ssl-certs/chain.crt
```

Step 6 — Seed the Database

```
python seeder.py
# Creates schema 'secops_db', users table, and default admin
# Default: admin@mail.com / password123  <- CHANGE IMMEDIATELY
```

Step 7 — Data Files

```
# Populate the UI data directory:
mkdir -p app/static/sources/vapt-img
# Copy / upload:
#   ec2-patch.xlsx
#   projects.xlsx
#   Accomplished_Activities.xlsx
#   vapt-img/*.png (or *.jpg)

# Populate the drive (reports browser) directory:
mkdir -p ../../source/drive   # 3 levels up from app/
# Place any PDF/DOCX/XLSX reports here
```

Step 8 — Set Port Capability & Start Service

```
sudo setcap 'cap_net_bind_service=+ep' /opt/secops-dash/venv/bin/python3

# Create startup script
nano /opt/secops-dash/SecopsDashboard.sh
chmod +x /opt/secops-dash/SecopsDashboard.sh

# Test manually first
/opt/secops-dash/SecopsDashboard.sh &
tail -f /var/log/secops-dash/$(date +"%d%m%y")-secops-dash.log

# Install systemd service
sudo cp prod/secops-dash.service /etc/systemd/system/
# Edit User=sdnch if your OS user is different
sudo systemctl daemon-reload
sudo systemctl enable --now secops-dash
```

12. Known Issues & Technical Debt

Issue	Severity	Detail
Hardcoded notifications	Medium	bell.py notifications are placeholder strings — need real data source integration
No email token for password reset	High	reset-password route allows changing any account's password without email verification — no token/link flow implemented
SSC cache is in-memory only	Low	Cache resets on every process restart; consider Redis or DB-backed cache for resilience
Duplicate requirements file	Low	app/requirements .txt (note the space in filename) is a stale copy — use root requirements.txt only
boto3 imported but unused	Low	AWS SDK is in requirements and imported — no actual S3 routes exist yet
Flask-Migrate configured but no migrations folder	Medium	Alembic/Flask-Migrate is installed but no migrations/ directory found — schema changes should use seeder or proper migration files
wsgi.py uses relative import	Medium	wsgi.py has 'from .app import create_app' — relative import only works when invoked as a package, not directly. Gunicorn invocation must be from project root

Issue	Severity	Detail
Live credentials in .env in ZIP	Critical	Rotate all credentials: POSTGRES_URI password, SSC_API_KEY, session_secret_key immediately

13. Version History

Version	Key Changes
V-3.0.20426 (Current)	Latest — clean up dead/unused codes, minor optimization of cache, migrated file storage to onedrive
V-2.9.20226	see .env for active version; WAF features migrated to Datadog, PDF viewer added, monitoring tool tabs added
V-2.8.25925	Tabs for open tools for monitoring. PDF file viewer. WAF features moved/migrated to Datadog. Navigation logic cleanup
V-2.0.28725	Headers/footers, version history, password reset, digital clock, basic notification module, data update timestamps, new data cards, WAF graph improvements, HTTPS/SSL security, codebase refactoring
V-1.5.25525	Reset Password modal in Admin UM, Edit User modal (role + active status), improved project cards UI, collapsible admin sidebar, fixed date parsing for Excel data, fixed scroll issue
V-1.0.25425	Initial release: Flask webapp, SecurityScorecard API integration, activity details, PostgreSQL bar chart (top 10 countries), 6-hour auto-refresh, role-based login, public homepage

14. Immediate Actions for Incoming Developer

The following tasks should be completed before taking ownership of this project:

- **CRITICAL: Rotate all credentials in .env — database password, SSC_API_KEY, and session_secret_key**
- **CRITICAL: Change the default admin password (admin@mail.com / password123) in the live database**
- **Implement email-token-based password reset flow to replace the current insecure self-service reset**
- Connect bell.py notifications to a real data source (DB table or external webhook)
- Set up a proper git repository for ongoing version control (the archive/git file is a bare repo log, not a full history)
- Create a Flask-Migrate migrations/ folder so schema changes are tracked and repeatable
- Verify SSL certificate expiry dates at /opt/ssl-certs/ and set up renewal reminders
- Confirm DASHBOARD_URLS in .env are still valid and embed the correct Datadog shared dashboard links

15. Contacts & Support

Application Support Email	Scipher.Support@Vestas.com
SSC API Docs	https://platform.securityscorecard.io/docs
Flask Docs	https://flask.palletsprojects.com/
SQLAlchemy Docs	https://docs.sqlalchemy.org/
Flask-Login Docs	https://flask-login.readthedocs.io/
Gunicorn Docs	https://gunicorn.org/

Handover Sign-Off

By signing below, both parties confirm that the handover of the Scipher Guardz SecOps Dashboard project has been completed and that the incoming developer has received all necessary access, documentation, and credentials.

Outgoing Developer	**Incoming Developer**
Name: _____C_Sudharshan_____	Name: ___Abirami L, Shivangi, Akash_____
Date: _____	Date: _____